

Dart Asynchronous Programming & Null Safety

Assignment 2 Overview

This assignment demonstrates the implementation of asynchronous operations using Dart's `Future`, `async`, and `await` keywords, alongside modern Null Safety principles and robust error handling patterns.

Dart Code Implementation

```
import 'dart:async';
```

```
class User {
```

```
  final int id;
```

```
  final String name;
```

```
  final String? email;
```

```
  final String? phone;
```

```
  final String? address;
```

```
  User({
```

```
required this.id,  
  
required this.name,  
  
this.email,  
  
this.phone,  
  
this.address,  
  
});  
  
factory User.fromJson(Map<String, dynamic> json) {  
  
  return User(  
  
    id: json['id'] as int? ?? 0,  
  
    name: json['name'] as String? ?? 'Anonymous User',  
  
    email: json['email'] as String?,  
  
    phone: json['phone'] as String?,  
  
    address: json['address'] as String?,  
  
  );  
  
}  
  
void displayInfo() {  
  
  print('👤 User ID: $id');
```

```
print('  Name: $name');

print('  Email: ${email ?? "Not provided"}');

print('  Phone: ${phone ?? "Not provided"}');

print('  Address: ${address ?? "Not provided"}');

}

}

class MockApiService {

  static Future<Map<String, dynamic>> fetchUserData(int userId) async {

    print('\n🌐 Fetching data for User ID: $userId from remote server...');

    await Future.delayed(const Duration(milliseconds: 1500));
    if (userId == 1) {

      return {

        'id': 1,

        'name': 'Prince Vaviya',

        'email': 'prince@example.com',

        'phone': '+91 98765 43210',

        'address': 'Surat, Gujarat',
```

```

    };

}

if (userId == 2) {

    return {

        'id': 2,

        'name': 'John Doe',

        'email': null,

        'phone': null,

    };

}

if (userId == 404) {

    throw Exception('HTTP 404: User with ID $userId not found on server.');
```

```

}

throw TimeoutException('Server took too long to respond. Connection timeout.');
```

```

}

}

Future<void> main() async {

    print('=====');
```

```

print('🚀 DART ASYNC, NULL SAFETY & ERROR HANDLING DEMO');

print('=====');

await loadAndDisplayUser(1);

await loadAndDisplayUser(2);

await loadAndDisplayUser(404);

await loadAndDisplayUser(999);

print('\n=====');

print('✅ All mock API requests completed safely!');

print('=====');

}

```

```

Future<void> loadAndDisplayUser(int userId) async {

  try {

    final Map<String, dynamic> data = await MockApiService.fetchUserData(userId);

    final user = User.fromJson(data);

    print('✅ Data fetched successfully:');

    user.displayInfo();
  } on TimeoutException catch (e) {
    print('🕒 Timeout Error: ${e.message}');
  }
}

```

```

} catch (e) {
  print('✖ Error occurred: $e');

} finally {
  print('--- Request cycle finished for ID $userId ---');

}

}

```

```

● princevaviya@Princes-MacBook-Air-3 dart_fundamentals % cd Assignment2
● princevaviya@Princes-MacBook-Air-3 Assignment2 % dart mock_api_fetcher.dart
=====
🚀 DART ASYNC, NULL SAFETY & ERROR HANDLING DEMO
=====

🌐 Fetching data for User ID: 1 from remote server...
✅ Data fetched successfully:
👤 User ID: 1
   Name: Prince Vaviya
   Email: prince@example.com
   Phone: +91 98765 43210
   Address: Surat, Gujarat
--- Request cycle finished for ID 1 ---

🌐 Fetching data for User ID: 2 from remote server...
✅ Data fetched successfully:
👤 User ID: 2
   Name: John Doe
   Email: Not provided
   Phone: Not provided
   Address: Not provided
--- Request cycle finished for ID 2 ---

🌐 Fetching data for User ID: 404 from remote server...
✖ Error occurred: Exception: HTTP 404: User with ID 404 not found on server.
--- Request cycle finished for ID 404 ---

🌐 Fetching data for User ID: 999 from remote server...
⌚ Timeout Error: Server took too long to respond. Connection timeout.
--- Request cycle finished for ID 999 ---

=====
✅ All mock API requests completed safely!
=====
○ princevaviya@Princes-MacBook-Air-3 Assignment2 % █

```

Conceptual Explanation

This code serves as a blueprint for robust communication between a client application and a remote server. The following analogies help clarify the core concepts:

1. Future & Async/Await: The Restaurant Order

- **Future (The Buzzer):** When you place an order at a restaurant, the staff gives you a buzzer. This buzzer represents a **Future**—a promise that your food will be ready eventually. You don't have it yet, but you hold the potential for it.
- **Async / Await:** This describes your behavior while waiting. You sit down, check your phone, or chat (the app continues running other tasks). When the buzzer vibrates, you **await** the result—you pause your other activities specifically to retrieve your meal.

2. Null Safety: The Optional Form

- **Null Safety:** Imagine a registration form where some fields are marked with an asterisk (required) and others are left empty (optional).
- **The Validation:** Dart's null safety is like a clerk checking that form. Using operators like **??** or **?**, the code ensures that if a field like "Email" is empty, the system provides a fallback (e.g., "Not provided") rather than crashing. It prevents the "billion-dollar mistake" of trying to read data that doesn't exist.

3. Error Handling: The Backup Plan

- **try:** You attempt to place your order and expect everything to go perfectly. This is the "Happy Path" where the kitchen has all the ingredients.
- **catch:** If the kitchen informs you they are out of ingredients or the oven is broken, you need a reaction. The **catch** block allows the app to handle these

"exceptions" gracefully—telling the user there was a connection error rather than simply stopping.

- **finally:** This represents the cleanup phase. Regardless of whether you ate a full meal or the kitchen was closed, you still need to return the buzzer and leave the table. It ensures resources are managed properly every time.